Lab Experiment 6: Logistic Regression and K Nearest Neighbours (KNN)

Aim: To implement Logistic Regression and KNN classifiers on the Breast Cancer Wisconsin (Diagnostic) dataset and compare their performance.

1. Import Libraries

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, ConfusionMatrixDisplay)

RANDOM_STATE = 42
sns.set_theme(style='whitegrid')
```

2. Load the Breast Cancer Dataset

The supplied `wdbc.data` file has no header row. The first column is an ID, the second column is the diagnosis, and the remaining columns are diagnostic measurements.

In [ ]:
```
feature_names = [
    'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean', 'smoothness_mean',
    'compactness_mean', 'concavity_mean', 'concave_points_mean', 'symmetry_mean', 'fractal
    'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
    'compactness_se', 'concavity_se', 'concave_points_se', 'symmetry_se', 'fractal_dimensi
    'radius_worst', 'texture_worst', 'perimeter_worst', 'area_worst', 'smoothness_worst',
    'compactness_worst', 'concavity_worst', 'concave_points_worst', 'symmetry_worst', 'fra
]
column_names = ['id', 'diagnosis'] + feature_names

df = pd.read_csv('wdbc.data', header=None, names=column_names)
print('Dataset shape:', df.shape)
display(df.head())
```

3. Exploratory Data Analysis

In [ ]:
```
print('Missing values in the dataset:', df.isnull().sum().sum())
print('Duplicate rows:', df.duplicated().sum())
display(df['diagnosis'].value_counts().rename_axis('Diagnosis').reset_index(name='Count'))

plt.figure(figsize=(6, 4))
sns.countplot(data=df, x='diagnosis', hue='diagnosis', palette=['#55a868', '#c44e52'], leg
plt.title('Number of Benign and Malignant Cases')
plt.xlabel('Diagnosis (B = Benign, M = Malignant)')
plt.ylabel('Number of Records')
plt.show()
```

Inference: The dataset contains 569 records, 30 diagnostic features, one ID column, and one diagnosis column. The ID is only an identifier, so it should not be used as a predictor. The class-count plot shows the balance between benign and malignant cases.

4. Data Preprocessing

`M` is encoded as 1 and `B` is encoded as 0. The data is split using stratification so both sets keep a similar class distribution.

In [ ]:
```
X = df.drop(columns=['id', 'diagnosis'])
y = df['diagnosis'].map({'B': 0, 'M': 1})

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print('Training set shape:', X_train_scaled.shape)
print('Testing set shape:', X_test_scaled.shape)
print('Training class distribution:')
print(y_train.value_counts(normalize=True).round(3))
```

Inference: There are no missing values, so no imputation is needed. Standardization gives every feature a similar scale, which is especially important for KNN because it uses distances between records. Scaling also helps Logistic Regression converge properly.

5. Train Logistic Regression and KNN Models

In [ ]:
```
logistic_model = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
knn_model = KNeighborsClassifier(n_neighbors=5)

logistic_model.fit(X_train_scaled, y_train)
knn_model.fit(X_train_scaled, y_train)

logistic_pred = logistic_model.predict(X_test_scaled)
```

```
knn_pred = knn_model.predict(X_test_scaled)

print('Both classifiers have been trained successfully.')
```

Inference: Logistic Regression learns a decision boundary using all features. KNN predicts a class by checking the five closest training records. Both methods use the same scaled training and testing data for a fair comparison.

6. Evaluate the Classifiers

In [ ]:
```
def get_metrics(y_true, y_pred):
    return {
        'Accuracy': accuracy_score(y_true, y_pred),
        'Precision': precision_score(y_true, y_pred, zero_division=0),
        'Recall': recall_score(y_true, y_pred, zero_division=0),
        'F1 Score': f1_score(y_true, y_pred, zero_division=0)
    }

comparison = pd.DataFrame(
    [get_metrics(y_test, logistic_pred), get_metrics(y_test, knn_pred)],
    index=['Logistic Regression', 'KNN (k=5)']
)
display(comparison.round(4))
```

In [ ]:
```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

ConfusionMatrixDisplay(confusion_matrix(y_test, logistic_pred), display_labels=['Benign',
    ax=axes[0], colorbar=False, cmap='Blues'
)
axes[0].grid(False)
axes[0].set_title('Logistic Regression')

ConfusionMatrixDisplay(confusion_matrix(y_test, knn_pred), display_labels=['Benign', 'Mali
    ax=axes[1], colorbar=False, cmap='Greens'
)
axes[1].grid(False)
axes[1].set_title('KNN (k=5)')

plt.tight_layout()
plt.show()
```

Inference: Accuracy gives the overall fraction of correct predictions. Precision shows how many predicted malignant cases are truly malignant, while recall shows how many actual malignant cases are found. F1 score combines precision and recall into one value.

7. Identify the Better Classifier

In [ ]:
```
best_model = comparison.sort_values(['F1 Score', 'Accuracy'], ascending=False).index[0]
print(f'Better classifier for this test split: {best_model}')
```

```
plt.figure(figsize=(8, 5))
comparison.plot(kind='bar', ylim=(0, 1.05), ax=plt.gca(), colormap='Set2')
plt.title('Performance Comparison of the Two Classifiers')
plt.xlabel('Classifier')
plt.ylabel('Score')
plt.xticks(rotation=0)
plt.legend(loc='lower right')
plt.show()
```

8. Conclusion

Inference: The comparison table and graph make it easy to compare both classifiers on the same test data. The model with the higher F1 score is selected here because it considers both precision and recall. The confusion matrices show the types of correct and incorrect predictions.

Conclusion: Both Logistic Regression and KNN can classify the diagnostic records after preprocessing. The model printed above is the better one for this test split based mainly on F1 score, with accuracy used as a tie-breaker.